NT-C-017 | C++ Programming | Course Knowledge Profile

Identity, purpose, prerequisites, scope and boundaries

Category	Programming
Target learners	Learners targeting C++ application or embedded-software foundations.
Prerequisites	Basic programming logic; C exposure is helpful.
Approved duration	8-10 weeks / 80-120 guided hours.
Final outcome	Build tested C++ applications using OOP, STL, memory management and algorithms.
Assessment profile	Programming/Platform

Approved tools / platforms

C++, modern STL, compiler/debugger, CMake basics, unit-testing framework, Git.

Knowledge boundaries

Include	Quality rule	Exclude / escalate
The eight approved modules, four sections, named tools, projects and mapped entry-level roles.	Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.	Do not treat copied code or syntax recall as proof of implementation skill.
Current product/platform procedures only when version and date are known.	Use primary documentation and record the source version.	Do not invent current commands, prices, tax rates, policies or certification status.
Evidence-based career guidance.	Cite tests, projects, capstone, viva and verified resume evidence.	No guaranteed job, placement, salary, interview or employer claim.

LLM answer pattern: identify the course and version, answer from approved records, cite record IDs, distinguish verified facts from guidance, and state any missing evidence.

NT-C-017 | C++ Programming | NT-C-017-S01 Deep Knowledge

Modules 1-2 | objectives, concepts, evidence and remediation

Section competency	Types, control, functions and references; Classes, constructors, inheritance and polymorphism
Completion evidence	Basic programs; OOP application
Section weight	20% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 1 - C++ foundation

Objective: Types, control, functions and references
Observable evidence: Basic programs

Concept	Approved knowledge statement	Application behaviour	Evidence
Types	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when types is appropriate; apply it in a C++ Programming task; verify the result.	Basic programs
control	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when control is appropriate; apply it in a C++ Programming task; verify the result.	Basic programs
functions	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when functions is appropriate; apply it in a C++ Programming task; verify the result.	Basic programs
references	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when references is appropriate; apply it in a C++ Programming task; verify the result.	Basic programs

Module 2 - Object-oriented C++

Objective: Classes, constructors, inheritance and polymorphism
Observable evidence: OOP application

Concept	Approved knowledge statement	Application behaviour	Evidence
Classes	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when classes is appropriate; apply it in a C++ Programming task; verify the result.	OOP application
constructors	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when constructors is appropriate; apply it in a C++ Programming task; verify the result.	OOP application
inheritance	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when inheritance is appropriate; apply it in a C++ Programming task; verify the result.	OOP application
polymorphism	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when polymorphism is appropriate; apply it in a C++ Programming task; verify the result.	OOP application

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Basic programs; OOP application, return to Modules 1-2, complete one guided practice and then use a new equivalent test form.

NT-C-017 | C++ Programming | NT-C-017-S02 Deep Knowledge

Modules 3-4 | objectives, concepts, evidence and remediation

Section competency	Pointers, RAII and smart pointers; Vector, map, set and iterators
Completion evidence	Safe resource exercise; Container application
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 3 - Memory and resources

Objective: Pointers, RAII and smart pointers
Observable evidence: Safe resource exercise

Concept	Approved knowledge statement	Application behaviour	Evidence
Pointers	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when pointers is appropriate; apply it in a C++ Programming task; verify the result.	Safe resource exercise
RAII	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when raii is appropriate; apply it in a C++ Programming task; verify the result.	Safe resource exercise
smart pointers	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when smart pointers is appropriate; apply it in a C++ Programming task; verify the result.	Safe resource exercise

Module 4 - STL containers

Objective: Vector, map, set and iterators
Observable evidence: Container application

Concept	Approved knowledge statement	Application behaviour	Evidence
Vector	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when vector is appropriate; apply it in a C++ Programming task; verify the result.	Container application
map	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when map is appropriate; apply it in a C++ Programming task; verify the result.	Container application
set	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when set is appropriate; apply it in a C++ Programming task; verify the result.	Container application
iterators	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when iterators is appropriate; apply it in a C++ Programming task; verify the result.	Container application

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Safe resource exercise; Container application, return to Modules 3-4, complete one guided practice and then use a new equivalent test form.

NT-C-017 | C++ Programming | NT-C-017-S03 Deep Knowledge

Modules 5-6 | objectives, concepts, evidence and remediation

Section competency	Searching, sorting and STL algorithms; Generic code and error handling
Completion evidence	Algorithm exercises; Reusable component
Section weight	25% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 5 - Algorithms

Objective: Searching, sorting and STL algorithms
Observable evidence: Algorithm exercises

Concept	Approved knowledge statement	Application behaviour	Evidence
Searching	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when searching is appropriate; apply it in a C++ Programming task; verify the result.	Algorithm exercises
sorting	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when sorting is appropriate; apply it in a C++ Programming task; verify the result.	Algorithm exercises
STL algorithms	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when stl algorithms is appropriate; apply it in a C++ Programming task; verify the result.	Algorithm exercises

Module 6 - Templates and exceptions

Objective: Generic code and error handling
Observable evidence: Reusable component

Concept	Approved knowledge statement	Application behaviour	Evidence
Generic code	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when generic code is appropriate; apply it in a C++ Programming task; verify the result.	Reusable component
error handling	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when error handling is appropriate; apply it in a C++ Programming task; verify the result.	Reusable component

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Algorithm exercises; Reusable component, return to Modules 5-6, complete one guided practice and then use a new equivalent test form.

NT-C-017 | C++ Programming | NT-C-017-S04 Deep Knowledge

Modules 7-8 | objectives, concepts, evidence and remediation

Section competency	I/O, unit testing and debugging; Architecture, build, Git and documentation
Completion evidence	Tested utility; Completed C++ project
Section weight	30% of the weighted mock-test average
Assessment	10 concept MCQ (30 marks) + 4 code/design scenarios (30) + 2 practical tasks (40)

Module 7 - Files and testing

Objective: I/O, unit testing and debugging
Observable evidence: Tested utility

Concept	Approved knowledge statement	Application behaviour	Evidence
I/O	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when i/o is appropriate; apply it in a C++ Programming task; verify the result.	Tested utility
unit testing	Systematic verification that expected behaviour and important failure cases work correctly. In C++ Programming, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when unit testing is appropriate; apply it in a C++ Programming task; verify the result.	Tested utility
debugging	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when debugging is appropriate; apply it in a C++ Programming task; verify the result.	Tested utility

Module 8 - Capstone

Objective: Architecture, build, Git and documentation
Observable evidence: Completed C++ project

Concept	Approved knowledge statement	Application behaviour	Evidence
Architecture	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when architecture is appropriate; apply it in a C++ Programming task; verify the result.	Completed C++ project
build	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when build is appropriate; apply it in a C++ Programming task; verify the result.	Completed C++ project
Git	Distributed version control for tracking and reviewing source changes. In C++ Programming, it must be demonstrated through an observable task, not only recalled as a definition.	Explain when git is appropriate; apply it in a C++ Programming task; verify the result.	Completed C++ project
documentation	A defined competency within C++ Programming. The learner must explain its purpose, apply it in an appropriate scenario, verify the result and produce the specified evidence.	Explain when documentation is appropriate; apply it in a C++ Programming task; verify the result.	Completed C++ project

Remediation: If the learner cannot explain the purpose, complete the stated task or provide Tested utility; Completed C++ project, return to Modules 7-8, complete one guided practice and then use a new equivalent test form.

NT-C-017 | C++ Programming | Skill Ontology & Dependency Map

Atomic skills, learning order and evidence strength

Skill ID	Competency	Depends on	Evidence	Type
NT-C-017-SK01	C++ foundation	Prerequisite foundation	Basic programs	Critical
NT-C-017-SK02	Object-oriented C++	M01 plus earlier foundations	OOP application	Critical
NT-C-017-SK03	Memory and resources	M02 plus earlier foundations	Safe resource exercise	Supporting
NT-C-017-SK04	STL containers	M03 plus earlier foundations	Container application	Supporting
NT-C-017-SK05	Algorithms	M04 plus earlier foundations	Algorithm exercises	Supporting
NT-C-017-SK06	Templates and exceptions	M05 plus earlier foundations	Reusable component	Critical
NT-C-017-SK07	Files and testing	M06 plus earlier foundations	Tested utility	Supporting
NT-C-017-SK08	Capstone	M07 plus earlier foundations	Completed C++ project	Critical

Evidence-strength hierarchy

Strength	Evidence source	Use
High	Trainer-observed practical, viva, working source/project files and reproducible output.	May support verified mastery when rubric threshold is met.
Medium	Scored mock tests, reviewed assignments and documented portfolio artifacts.	Supports mastery with corroboration.
Low	Resume claim, self-report, attendance or video completion without demonstration.	May trigger verification; cannot alone support Apply Now.

CareerTours must not treat attendance, time spent or content opened as skill mastery.

NT-C-017 | C++ Programming | Assessment Knowledge & Sample Items

Non-live examples for LLM training and evaluation

Sample ID	Type	Question	Approved answer / rubric anchor
NT-C-017-S01-EX01	Evidence/application	What evidence best demonstrates the combined competency of C++ foundation and Object-oriented C++?	The learner should provide Basic programs; OOP application and explain how the work applies Types, control, functions and references; Classes, constructors, inheritance and polymorphism. Recall alone is insufficient.
NT-C-017-S01-EX02	Scenario/remediation	A learner reports completing Object-oriented C++ but cannot produce OOP application. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-017-S01, request OOP application, and allow a new equivalent test only after remediation.
NT-C-017-S02-EX01	Evidence/application	What evidence best demonstrates the combined competency of Memory and resources and STL containers?	The learner should provide Safe resource exercise; Container application and explain how the work applies Pointers, RAII and smart pointers; Vector, map, set and iterators. Recall alone is insufficient.
NT-C-017-S02-EX02	Scenario/remediation	A learner reports completing STL containers but cannot produce Container application. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-017-S02, request Container application, and allow a new equivalent test only after remediation.
NT-C-017-S03-EX01	Evidence/application	What evidence best demonstrates the combined competency of Algorithms and Templates and exceptions?	The learner should provide Algorithm exercises; Reusable component and explain how the work applies Searching, sorting and STL algorithms; Generic code and error handling. Recall alone is insufficient.
NT-C-017-S03-EX02	Scenario/remediation	A learner reports completing Templates and exceptions but cannot produce Reusable component. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-017-S03, request Reusable component, and allow a new equivalent test only after remediation.
NT-C-017-S04-EX01	Evidence/application	What evidence best demonstrates the combined competency of Files and testing and Capstone?	The learner should provide Tested utility; Completed C++ project and explain how the work applies I/O, unit testing and debugging; Architecture, build, Git and documentation. Recall alone is insufficient.
NT-C-017-S04-EX02	Scenario/remediation	A learner reports completing Capstone but cannot produce Completed C++ project. What should CareerTours do?	Mark the evidence Not Yet Verified, assign practice from NT-C-017-S04, request Completed C++ project, and allow a new equivalent test only after remediation.

Live bank rule: Generate at least three times the live quantity for each section using the approved Programming/Platform blueprint. Human-review every scored item before initial production release.

NT-C-017 | C++ Programming | Labs, Projects & Scoring Rubrics

Practical proof of learning and ownership

Level	Approved project	Skills evidenced	Required deliverables
Mini 1	Inventory console app	OOP and STL	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Mini 2	Log analyser	Files, algorithms and errors	Brief; working/source file or reproducible environment; output; validation notes; reflection.
Capstone	C++ management system	Architecture, tests and documentation	Brief; working/source file or reproducible environment; output; validation notes; reflection.

Standard project rubric

Dimension	Weight	What earns credit
Problem framing	15%	Clear objective, user/business need, assumptions and success criteria.
Technical/design accuracy	25%	Correct methods, appropriate tools and sound decisions.
Completeness	15%	Required features, assets, data or workflow are present.
Testing and validation	15%	Important normal, edge, quality or failure cases are checked.
Evidence and reproducibility	15%	Source/working files, setup, inputs, outputs and version information are available.
Documentation	5%	Concise structure, instructions and limitations.
Viva / ownership	10%	Learner can explain decisions, demonstrate work and respond to a change request.

Prefer runnable or demonstrable outputs, clear input validation, error handling, tests and version control.

NT-C-017 | C++ Programming | Career Roles, Evidence & Gap Actions

Explainable top-three occupation matching

Rank	Occupation	Default weighted skills	Minimum evidence	Gap action
1	Junior C++ Programmer	OOP 34% [critical]; STL 33% [critical]; Git 33%	Tested repository	If a critical skill is below 60 or tested repository is missing, return Improve First or Needs Evidence with the linked module/project.
2	Software Development Trainee - C++	Algorithms 34% [critical]; debugging 33% [critical]; build 33%	Working application	If a critical skill is below 60 or working application is missing, return Improve First or Needs Evidence with the linked module/project.
3	Embedded/Application Trainee	Memory 34% [critical]; C++ 33% [critical]; diagnostics 33%	Capstone evidence	If a critical skill is below 60 or capstone evidence is missing, return Improve First or Needs Evidence with the linked module/project.

Course readiness	45% section mocks + 20% mini projects + 25% capstone + 10% viva.
Role readiness	45% role-relevant mastery + 25% projects/capstone + 20% aligned mocks + 10% verified resume evidence.
Evidence confidence	Show separately; Apply Now requires at least 70.
Resume extraction	Use only relevant skills, projects, education, experience and portfolio evidence; minimise personal data before external model processing.

A student may be Apply Now for one mapped occupation and Improve First for another. Return independent evidence and gaps for each role.

NT-C-017 | C++ Programming | FAQs, Misconceptions & LLM Response Pack

Course-specific retrieval and response patterns

Approved FAQ	Answer
Who is C++ Programming for?	Learners targeting C++ application or embedded-software foundations.
What should I know before starting?	Basic programming logic; C exposure is helpful.
How long is the approved learning journey?	8-10 weeks / 80-120 guided hours.
What will I be able to demonstrate?	Build tested C++ applications using OOP, STL, memory management and algorithms.
Does completion guarantee a job?	No. Completion and role readiness are evidence-based guidance; employment, placement, salary and employer selection are not guaranteed.
What if I fail a section?	CareerTours identifies the failed skill tags and module, assigns targeted practice, and permits an equivalent retest after remediation.

Misconception	Approved correction
Attendance proves mastery	False - require tests and observable practical/project evidence.
One total score proves every role fit	False - calculate each occupation using its weighted skills and evidence.
The LLM can add missing details	False - retrieve approved records or state that evidence is insufficient.
Course completion guarantees employment	False - never claim guaranteed job, placement, salary or interview.
C++ Programming knowledge never changes	False - version-sensitive tools, laws, taxes, security and platform procedures require review.

Retrieval metadata

```
course_code=NT-C-017; corpus_version=3.0;
record_types=course_profile,module,section,skill,assessment_sample,project,occupation_mapping,faq,misconception;
approval_status=approved; effective_date=2026-08-12
```